

GitHub REST API Reference - Combined (v2022-11-28 and v2025-06-01)

Contents

Render a Markdown document	1
Render a Markdown document	3
Create an issue	4
Create an issue	5
Create a pull request	6
Create a pull request	7
Create a repository for the authenticated user	8
Create a repository for the authenticated user	9
Create a release	10
Create a release	12

Render a Markdown document

Use the REST API to render a Markdown document as an HTML page or as raw text. This endpoint accepts a JSON body describing the text to render and how to render it.

Request

```
curl -L \
  -X POST \
  -H "Accept: application/vnd.github+json" \
  -H "Authorization: Bearer <YOUR-TOKEN>" \
  -H "X-GitHub-API-Version: 2022-11-28" \
  https://api.github.com/markdown \
  -d '{"text":"Hello **world**"}'
```

Body parameters

Name	Type	Required	Description
text	string	Yes	The Markdown text to render in HTML.
mode	string	No	The rendering mode. Default: markdown. Can be one of: markdown, gfm.
context	string	No	The repository context to use when creating references in gfm mode.

Response

```
"<p>Hello <strong>world</strong></p>"
```

Notes

In raw mode, GitHub Flavored Markdown is not supported and Markdown is rendered in plain format, similar to a README.md file. Markdown content must be 400 KB or less.

Render a Markdown document

Applies to API version 2025-06-01 and later.

Renders a Markdown document server-side, returning either an HTML fragment or raw text depending on the requested mode.

Two changes from earlier API versions: the context field now accepts an owner/repo pair as a structured object rather than a plain string, and a new `wrap_tables` flag controls whether wide tables are wrapped for narrow viewports.

```
curl -L \
  -X POST \
  -H "Accept: application/vnd.github+json" \
  -H "Authorization: Bearer <YOUR-TOKEN>" \
  -H "X-GitHub-API-Version: 2025-06-01" \
  https://api.github.com/markdown \
  -d '{"text":"Hello **world**","wrap_tables":true}'
```

Parameters accepted in the request body:

- `text` (string, **required**) — The Markdown text to render.
- `mode` (string, optional, default `markdown`) — One of `markdown` or `gfm`.
- `context` (object, optional) — { `"owner": "...", "repo": "...` }. Used for `gfm`-mode reference linking.
- `wrap_tables` (boolean, optional, default `false`) — When `true`, wide tables render with horizontal scroll wrappers instead of overflowing.

Response body is an HTML string (or raw text, in raw mode). Content must be 400 KB or less.

Previous version (2022-11-28)

The prior version of this endpoint accepted only `text`, `mode`, and `context`, where `context` was a plain string (e.g. `"octocat/Hello-World"`) rather than a structured object. There was no `wrap_tables` option in that version — wide tables in rendered output simply overflowed their container with no wrapping behavior available.

Create an issue

POST /repos/{owner}/{repo}/issues

Any user with pull access to a repository can create an issue.

Parameters

Name	In	Type	Description
owner	path	string	Required. The account owner of the repository.
repo	path	string	Required. The name of the repository without the .git extension.
title	body	string or integer	Required. The title of the issue.
body	body	string	The contents of the issue.
assignee	body	string	Login for the user that this issue should be assigned to.
labels	body	array of strings	Labels to associate with this issue.

Example request

```
curl -L \
  -X POST \
  -H "Accept: application/vnd.github+json" \
  https://api.github.com/repos/octocat/Hello-World/issues \
  -d '{"title":"Found a bug","body":"I\'m having a problem."}'
```

Example response

```
{
  "id": 1,
  "number": 1347,
  "state": "open",
  "title": "Found a bug"
}
```

Create an issue

POST /repos/{owner}/{repo}/issues

API version 2025-06-01

Creates an issue on the specified repository. Any user with pull access can create one.

Starting in this version, assignee is deprecated in favor of assignees (array), and a new optional type field lets you set the issue type at creation time instead of via a separate call.

Path parameters

Name	Type	Description
owner	string	The account owner of the repository.
repo	string	The name of the repository.

Body parameters

Name	Type	Required	Default	Description
title	string or integer	Yes	—	The title of the issue.
body	string	No	null	The contents of the issue.
assignees	array of strings	No	[]	Logins for the users assigned to this issue. Replaces the deprecated assignee field.
labels	array of strings	No	[]	Labels to associate with this issue.
type	string	No	null	The name of the issue type to set on creation.

```
curl -L -X POST \
  -H "X-GitHub-API-Version: 2025-06-01" \
  https://api.github.com/repos/octocat/Hello-World/issues \
  -d '{"title":"Found a bug","assignees":["octocat"],"type":"Bug"}
```

Previous version (2022-11-28)

The prior version of this endpoint only accepted a single assignee (string) rather than an assignees array, so an issue could only be assigned to one user at creation time — assigning multiple users required a separate follow-up call. There was also no type field; setting an issue type required a second request after creation.

Create a pull request

Draft pull requests are available in public repositories with GitHub Free and GitHub Free for organizations, and in public and private repositories with GitHub Pro, GitHub Team and GitHub Enterprise Cloud. For more information, see the [GitHub products page](#).

To create a pull request you provide a title, base and head branches, and optionally a body describing the change.

Endpoint: POST /repos/{owner}/{repo}/pulls

Parameters:

- title — string — required — The title of the new pull request.
- head — string — required — The name of the branch where your changes are implemented.
- base — string — required — The name of the branch you want the changes pulled into.
- body — string — optional — The contents of the pull request.
- draft — boolean — optional, default false — Indicates whether the pull request is a draft.

```
curl -X POST https://api.github.com/repos/octocat/Hello-World/pulls \
  -d '{"title":"Amazing feature","head":"octocat:new-feature","base":"master"}
```

Returns a pull request object on success (status 201).

Create a pull request

Version 2025-06-01 · [reference](#)

Overview

Opens a pull request against the given repository. Since 2025-06-01, the API supports specifying `maintainer_can_modify` at creation time (previously this required a follow-up PATCH call), and draft pull requests can be created directly as ready-for-review by omitting the field entirely — its default has not changed but its interaction with auto-merge settings has.

Path parameters

Name	Type
owner	string
repo	string

Request body

Field	Type	Required	Notes
title	string	Yes	Required unless issue is specified.
head	string	Yes	Branch or user:branch for cross-repo.
base	string	Yes	The branch you want changes merged into.
body	string	No	PR description.
draft	boolean	No	Default false.
maintainer_can_modify	boolean	No	Default true. Now settable at creation time.

Example

```
curl -X POST -H "X-GitHub-API-Version: 2025-06-01" \
  https://api.github.com/repos/octocat/Hello-World/pulls \
  -d '{"title":"Amazing feature",
      "head":"octocat:new-feature",
      "base":"master",
      "maintainer_can_modify":false}'
```

Returns 201 Created with the pull request object.

Previous version (2022-11-28)

`maintainer_can_modify` could not be set at creation time in the prior version — it was true by default with no way to override it until a follow-up PATCH request. The interaction between draft and auto-merge settings also behaved differently: a draft PR could not have auto-merge enabled until explicitly marked ready for review, whereas this version allows auto-merge to be pre-armed on a draft.

Create a repository for the authenticated user

POST /user/repos

Creates a new repository for the authenticated user.

OAuth scope required: public_repo or repo scope.

Parameter	Type	Required	Description
name	string	yes	The name of the repository.
description	string	no	A short description of the repository.
homepage	string	no	A URL with more information about the repository.
private	boolean	no	Whether the repository is private. Default: false.
has_issues	boolean	no	Whether issues are enabled. Default: true.
auto_init	boolean	no	Pass true to create an initial commit with empty README. Default: false.

```
curl -X POST https://api.github.com/user/repos \
-d '{"name":"Hello-World","private":true}'
```

Create a repository for the authenticated user

POST /user/repos — API version 2025-06-01

What changed

As of this version, `has_issues` defaults to true only for public repositories; private repositories now default `has_issues` to false unless explicitly set. A new visibility enum field supersedes the boolean `private` for organizations offering internal repositories, though `private` continues to work for personal accounts.

Parameters

- **name** (string, required) — Repository name.
- **description** (string, optional) — Short repository description.
- **visibility** (enum, optional) — One of public, private, internal. Default: public.
- **private** (boolean, optional, deprecated in favor of visibility) — Default: false.
- **has_issues** (boolean, optional) — Default depends on visibility: true for public, false for private/internal.
- **auto_init** (boolean, optional) — Default: false. Creates an initial commit with an empty README when true.

Request

```
curl -X POST -H "X-GitHub-API-Version: 2025-06-01" \
  https://api.github.com/user/repos \
  -d '{"name":"Hello-World","visibility":"internal","auto_init":true}'
```

Response

Returns 201 Created with the full repository object.

Previous version (2022-11-28)

The prior version only had the boolean `private` field — there was no visibility enum, and organizations offering “internal” visibility had no way to request it through this endpoint. `has_issues` also defaulted to true unconditionally, regardless of whether the repository was public or private.

Create a release

Users with push access to the repository can create a release.

This endpoint triggers notifications. Creating content too quickly using this endpoint may result in secondary rate limiting.

Parameters for “Create a release”

Path parameters

Name	Type	Description
owner	string	The account owner of the repository.
repo	string	The name of the repository.

Body parameters

A release requires a git tag to point to. If the tag already exists, this endpoint attaches the release to that tag; otherwise a lightweight tag with that name is created automatically at the target commitish when the release is published, and this automatic behavior is one of the more commonly misunderstood parts of the release flow because many integrators assume they must create the tag themselves beforehand via the Git Data API, when in fact simply supplying tag_name against a target_commitish that does not yet carry that tag is sufficient and is in fact the more common pattern seen across the majority of CI/CD release pipelines that call this endpoint directly after a successful build, since it saves a full extra round trip through the separate tags endpoint and reduces the chance of a race condition between tag creation and release creation.

Name	Type	Required	Description
tag_name	string	Yes	The name of the tag.
target_commitish	string	No	Specifies the commitish value that determines where the Git tag is created from. Default: the repository's default branch.
name	string	No	The name of the release.
body	string	No	Text describing the contents of the tag.
draft	boolean	No	true makes the release a draft. Default: false.
prerelease	boolean	No	true identifies the release as a prerelease. Default: false.

Name	Type	Required	Description
generate_release_notes	boolean	No	Whether to automatically generate the name and body for this release. Default: false.

```
curl -L -X POST https://api.github.com/repos/octocat/Hello-World/releases \
-d '{"tag_name":"v1.0.0",
    "target_commitish":"master",
    "name":"v1.0.0",
    "draft":false,
    "prerelease":false}'
```

Returns 201 Created with the release object.

Create a release

API version 2025-06-01

Summary

Publishes a new release against an existing or new tag. As of this version, `discussion_category_name` can be supplied to automatically open a linked discussion, and `make_latest` gives explicit control over whether the release becomes the repository's "Latest" release, replacing an older undocumented heuristic based purely on publish date.

Body parameters

Name	Type	Required	Default	Description
<code>tag_name</code>	string	Yes	—	The name of the tag.
<code>target_commitish</code>	string	No	default branch	Commitish value the tag is created from, if it doesn't already exist.
<code>name</code>	string	No	null	Release title.
<code>body</code>	string	No	null	Release notes body text.
<code>draft</code>	boolean	No	false	Creates a draft (unpublished) release when true.
<code>prerelease</code>	boolean	No	false	Marks the release as a prerelease.
<code>discussion_category_name</code>	string	No	null	If specified, a discussion of the specified category is created and linked to the release.
<code>generate_release_notes</code>	boolean	No	false	Auto-generates name/body from merged PRs since the previous release.
<code>make_latest</code>	enum	No	true	One of true, false, legacy. Controls whether this release is marked "Latest".

Example

```
curl -L -X POST -H "X-GitHub-API-Version: 2025-06-01" \
  https://api.github.com/repos/octocat/Hello-World/releases \
  -d '{"tag_name":"v2.0.0","make_latest":"true","generate_release_notes":true}'
```

Returns 201 Created.

Previous version (2022-11-28)

The prior version had no `discussion_category_name` field, so linking a discussion to a release required a separate manual step in the UI after the release was published. There was also no `make_latest` control — whether a release showed as “Latest” was determined purely by an internal heuristic based on publish date and prerelease/draft status, which occasionally marked an older release as “Latest” if a newer one was still in draft.